

# Unit Tests for Simulation Framework

## Overview

This test suite provides comprehensive unit tests for the discrete event simulation framework, covering all major modules:

- **Core:** Entity, EventLogger, BaseBlock, SimulationModel
- **Blocks:** CreateBlock, ProcessBlock, DecideBlock, DisposeBlock
- **Validation:** StabilityAnalyzer, WarmUpAnalyzer, ResourceValidator
- **Analytics:** MetricsCollector, WIPTracker, FinancialAnalyzer, SimulationReporter, SimulationPlotter
- **Statistics:** ReplicationFramework, FactorialExperiment
- **Config:** SimulationConfig
- **Integration:** End-to-end model tests

## Test Structure

```
tests/
├── __init__.py
├── confest.py          # Shared fixtures
├── test_core/
│   ├── __init__.py
│   ├── test_entity.py    # Entity, EventLogger tests
│   ├── test_base_block.py # BaseBlock tests
│   └── test_simulation_model.py # SimulationModel tests
├── test_blocks/
│   ├── __init__.py
│   ├── test_create_block.py # CreateBlock tests
│   ├── test_process_block.py # ProcessBlock tests
│   ├── test_decide_block.py # DecideBlock tests
│   └── test_dispose_block.py # DisposeBlock tests
├── test_validation/
│   ├── __init__.py
│   ├── test_stability.py    # StabilityAnalyzer tests
│   ├── test_warmup.py      # WarmUpAnalyzer tests
│   └── test_resource_validator.py # ResourceValidator tests
├── test_analytics/
│   ├── __init__.py
│   └── test_metrics.py      # MetricsCollector tests
```

```
| | └─ test_wip_metrics.py      # WIPTracker tests
| | └─ test_financial.py       # FinancialAnalyzer tests
| | └─ test_reporting.py       # SimulationReporter tests
| | └─ test_plotting.py        # SimulationPlotter tests
| └─ test_statistics/
| | └─ __init__.py
| | └─ test_replication.py     # ReplicationFramework tests
| | └─ test_factorial.py       # FactorialExperiment tests
| └─ test_config/
| | └─ __init__.py
| | └─ test_simulation_config.py # SimulationConfig tests
└─ test_integration/
    └─ __init__.py
    └─ test_simple_model.py     # Simple queue integration tests
    └─ test_priority_model.py   # Priority queuing tests
    └─ test_validation_integration.py # Validation integration tests
```

## Installation

### Install Required Packages

```
bash

pip install pytest pytest-cov
```

### Install Simulation Framework Dependencies

```
bash

pip install simpy numpy scipy pandas matplotlib seaborn
```

## Running Tests

### Run All Tests

```
bash

pytest tests/ -v
```

### Run Specific Test Module

```
bash
```

```
# Test core module
```

```
pytest tests/test_core/ -v
```

```
# Test blocks module
```

```
pytest tests/test_blocks/ -v
```

```
# Test validation module
```

```
pytest tests/test_validation/ -v
```

```
# Test analytics module
```

```
pytest tests/test_analytics/ -v
```

```
# Test statistics module
```

```
pytest tests/test_statistics/ -v
```

## Run Specific Test File

```
bash
```

```
pytest tests/test_core/test_entity.py -v
```

## Run Specific Test Class

```
bash
```

```
pytest tests/test_core/test_entity.py::TestEntity -v
```

## Run Specific Test Function

```
bash
```

```
pytest tests/test_core/test_entity.py::TestEntity::test_entity_creation -v
```

## Coverage Reports

### Generate HTML Coverage Report

```
bash
```

```
pytest tests/ --cov=. --cov-report=html
```

This creates an `htmlcov/` directory with detailed coverage reports. Open `htmlcov/index.html` in your browser.

# Generate Terminal Coverage Report

```
bash

pytest tests/ --cov=. --cov-report=term-missing
```

## Coverage for Specific Module

```
bash

pytest tests/test_core/ --cov=core --cov-report=term-missing
```

## Test Categories

### 1. Unit Tests

Test individual components in isolation:

```
bash

pytest tests/test_core/ tests/test_blocks/ -v
```

### 2. Integration Tests

Test complete workflows:

```
bash

pytest tests/test_integration/ -v
```

### 3. Validation Tests

Test error detection and validation:

```
bash

pytest tests/test_validation/ -v
```

## Key Testing Patterns

### Using Fixtures

```
python
```

```
def test_with_model(simple_model):  
    """Test using shared model fixture."""  
    assert simple_model.blocks == {}
```

## Testing Exceptions

```
python  
  
def test_raises_error():  
    """Test that error is raised."""  
    with pytest.raises(ValueError):  
        # Code that should raise ValueError  
        pass
```

## Testing Output

```
python  
  
def test_print_output(capsys):  
    """Test printed output."""  
    print("Hello")  
    captured = capsys.readouterr()  
    assert "Hello" in captured.out
```

## Testing File Operations

```
python  
  
def test_file_export(tmp_path):  
    """Test file export using temporary directory."""  
    file_path = tmp_path / "test.csv"  
    # Write to file_path  
    assert file_path.exists()
```

## Parametrized Tests

```
python
```

```
@pytest.mark.parametrize("capacity,units,should_pass", [
    (3, 2, True), #units < capacity
    (3, 3, False), #units == capacity (warning)
    (3, 4, False), #units > capacity (error)
])
def test_resource_validation(capacity, units, should_pass):
    """Test resource validation with different configurations."""
    # Test logic here
    pass
```

## Test Execution Options

### Run Tests in Parallel

```
bash

pip install pytest-xdist
pytest tests/ -n auto # Use all CPU cores
```

### Run Only Failed Tests

```
bash

pytest tests/ --lf # Last failed
pytest tests/ --ff # Failed first
```

### Stop on First Failure

```
bash

pytest tests/ -x
```

### Show Local Variables on Failure

```
bash

pytest tests/ -l
```

### Verbose Output

```
bash
```

```
pytest tests/ -vv # Very verbose
```

## Quiet Output

```
bash
```

```
pytest tests/ -q
```

## Continuous Integration

### Example GitHub Actions Workflow

```
yaml

name: Tests

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Set up Python
        uses: actions/setup-python@v2
        with:
          python-version: 3.9
      - name: Install dependencies
        run: |
          pip install pytest pytest-cov
          pip install simpy numpy scipy pandas matplotlib seaborn
      - name: Run tests
        run: |
          pytest tests/ --cov=. --cov-report=xml
      - name: Upload coverage
        uses: codecov/codecov-action@v2
```

## Writing New Tests

### Template for New Test File

```
python
```

```

# tests/test_module/test_new_component.py
import pytest
from module.new_component import NewComponent

class TestNewComponent:
    """Test NewComponent functionality."""

    def test_initialization(self):
        """Test creating a new component."""
        component = NewComponent()
        assert component is not None

    def test_basic_operation(self):
        """Test basic component operation."""
        component = NewComponent()
        result = component.operate()
        assert result == expected_value

    def test_edge_case(self):
        """Test edge case handling."""
        component = NewComponent()
        with pytest.raises(ValueError):
            component.invalid_operation()

```

## Best Practices

1. **One assertion per test** (when possible)
2. **Clear test names** describing what is being tested
3. **Arrange-Act-Assert** pattern
4. **Use fixtures** for common setup
5. **Test edge cases** and error conditions
6. **Mock external dependencies** when needed
7. **Keep tests independent** (no test dependencies)
8. **Test both success and failure** paths

## Troubleshooting

**ImportError: No module named 'core'**

Make sure the project root is in PYTHONPATH:

```
bash
export PYTHONPATH="${PYTHONPATH}:${pwd}"
pytest tests/
```

Or install in development mode:

```
bash
pip install -e .
```

## Tests Hanging

Some simulation tests may hang if there's a deadlock. Use timeouts:

```
bash
pytest tests/ --timeout=10 # 10 second timeout per test
```

Requires: `pip install pytest-timeout`

## Random Test Failures

If tests fail randomly, they may have race conditions or depend on random values. Use fixed seeds:

```
python
def test_with_seed(seed_value):
    """Test using fixed seed fixture."""
    random.seed(seed_value)
    # Test logic
```

## Coverage Goals

- **Core modules:** 90%+ coverage
- **Blocks:** 85%+ coverage
- **Validation:** 90%+ coverage
- **Analytics:** 80%+ coverage (plotting functions may be lower)
- **Statistics:** 85%+ coverage

- **Overall:** 85%+ coverage

## Additional Tools

### Code Quality

```
bash

# Linting
pip install pylint
pylint simulation_framework/

# Type checking
pip install mypy
mypy simulation_framework/

# Code formatting
pip install black
black simulation_framework/ tests/
```

### Test Markers

```
python

@pytest.mark.slow
def test_long_simulation():
    """Test marked as slow."""
    pass

@pytest.mark.integration
def test_full_model():
    """Test marked as integration."""
    pass
```

Run only specific markers:

```
bash

pytest tests/ -m "not slow" # Skip slow tests
pytest tests/ -m integration # Only integration tests
```

## Summary

This comprehensive test suite ensures:

- ✓ **Correctness:** All components work as expected
- ✓ **Reliability:** Edge cases and errors are handled
- ✓ **Maintainability:** Changes don't break existing functionality
- ✓ **Documentation:** Tests serve as usage examples
- ✓ **Confidence:** Safe refactoring and feature addition

Run tests regularly during development and before commits!